Zombie Hunter — Architecture

GCP, the walking dead.

Purpose: find forgotten test/dev projects ("zombies") in a GCP organization and produce an evidence-cited assessment — without ever risking a project that still matters, and without any automated path to deletion.

Tool project	`sdx-demos-zombie-dev` (Enterprise/Dev folder), region `us-central1`
Repo	`sequin-au/zombie-hunter` (private)
Infrastructure	Terraform in-repo at `terraform/` (vendored project-factory module; untracked backend/tfvars)
Portal	Cloud Run built-in IAP on the run.app URL (no LB, no Cloud Armor)
Schedule	Daily 06:00 Australia/Sydney (Cloud Scheduler → pipeline job)
Status	Deployed 2026-07-13; runtime shapes TF-codified 2026-07-15

---

1. The core idea: zombies are self-contained, not silent

A zombie is not a project with *zero* activity — dead projects still emit machine heartbeats (cron jobs, log writes, monitoring agents) and still incur spend (disks, static IPs, storage). A zombie is a project whose activity is **self-contained**: it costs money, it does things, but nothing outside it would notice if it vanished.

That framing drives the three assessment pillars:

Pillar	Weight	Signal
Billing	0.35	Spend **composition** (share of pure holding-cost SKUs: storage, static IPs, snapshots) and **variance** (perfectly flat month-on-month spend beats *low* spend as a death signal)
Last accessed	0.40	Last **human** admin action from Admin Activity audit logs (400-day retention) — service-account writes are deliberately excluded; machine heartbeat ≠ life
Connectivity	0.25	Cross-project dependency surface below the veto threshold

Each pillar scores 0–100 (100 = deadest) with a confidence level, and the weighted sum maps to a verdict:

- **zombie-high** — score ≥ 80, no vetoes
- **zombie-medium** — score ≥ 55, no vetoes
- **investigate** — *any* veto, regardless of score (hard rule)
- **active** — score < 55, or passed triage

All thresholds, weights, and veto definitions live in a versioned policy file (`config/policy.yaml`, currently `2026-07-17.2`) — **policy-as-file, never in prompts**. Verdict logic is deterministic code; the LLM cannot change it.

2. The eight hard vetoes

The scariest failure mode is deleting a project that something else secretly depends on. Eight structural signals therefore force `investigate` no matter how dead the project scores:

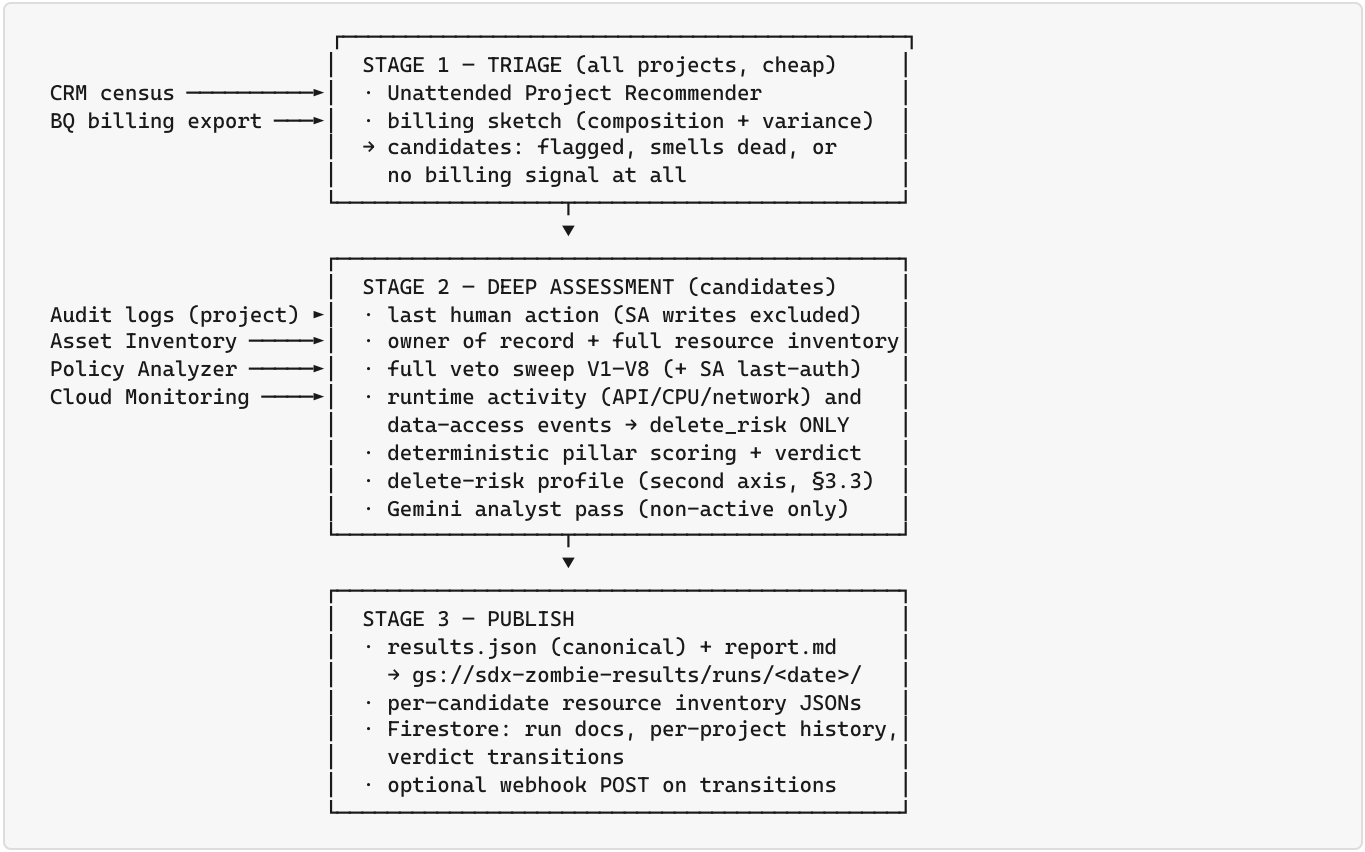
	Veto	Why it blocks
V1	Service account used from / granted in other projects — corroborated with Policy Analyzer SA last-authentication (actively exercised vs dormant grant)	Deleting kills someone else's auth
V2	CMEK keys potentially encrypting data outside the project	Deleting destroys the keys → **permanent** data loss elsewhere
V3	Shared VPC host (subnet grants to external principals), VPC peering, Private Service Connect, VPN tunnels, custom routes	Network fabric others transit
V4	OAuth / IAP clients	Deleting breaks logins elsewhere
V5	Destination for log sinks, exports, Terraform state, DNS zones	Other systems write here
V6	Artifact Registry images pulled by outside principals	Other deploys pull from here
V7	Resource liens or `retain=true` label	Explicit "do not touch"
V8	API keys bound to service accounts (`serviceAccountEmail` on the key)	External callers authenticate through them; unbound keys are evidence-grade only

Two guards prevent false positives on *young* projects:

- **Tenure guard** — projects younger than 60 days are never scored as zombies (bypassed only by the `demo-zombie=true` seed label).
- **Provisioning grace** — human actions within 48 h of project creation are setup, not usage, and don't count as "life".

3. Two-stage pipeline

Assessing every project deeply doesn't scale (and is the production scaling story for this demo). The pipeline is a Cloud Run Job (`zombie-pipeline`, 1 vCPU / 1 GiB, 60 min timeout) with three stages:



3.1 From first sighting to declared zombie — the gauntlet

"Zombie" is never a one-shot call. A project passes through six ordered gates, each of which can only stop or soften the verdict — and even a full zombie-high verdict is advice for a human workflow, not an action:

Gate 1 — census exclusions. The tool project and the bootstrap seed project are never assessed. Everything else in the org enters triage.

Gate 2 — triage (does it even *smell* dead?). A project is only deep- assessed if the Unattended Project Recommender flagged it, or its spend profile smells dead (>80 % holding-cost spend that is either flat — variance <0.1 — or too young to show variance, <2 billed months; or zero spend), or there is **no billing signal at all** — absence of evidence sends it to a closer look rather than a pass. Everything else is verdicted `active` ("passed triage") on the spot and no deep collectors run.

Gate 3 — initial pillar state. Each pillar starts from the collectors' evidence, and *missing signals score neutral, never deadly*:

- *Billing:* no export data → **50 / unknown** (neutral). Zero spend → 90 / medium ("either dead or free-tier"). Otherwise the score is the holding-cost percentage, +30 if spend is dead-flat (variance <0.05) across ≥2 months, scaled ×0.8.
- *Last accessed:* no human admin action in the 400-day audit retention → **100 / high** — the strongest single zombie signal. A human action that falls within 48 h of project creation scores 90 ("only

provisioning-era activity" — a forgotten project's only human touch is its own creation). Anything more recent scales the score down toward 0.
- *Connectivity:* no cross-project surface → 95. Low-grade signals (AR repos, KMS keys that didn't trip a veto) → 70 / medium. Incomplete veto checks *lower confidence* and shave a few points — a lesson from a live bug where a transient check failure flattened distinct projects onto one identical score.

Gate 4 — deterministic verdict, guards first. The weighted score (0.35 / 0.40 / 0.25) and overall confidence (worst pillar wins) are computed, then applied in strict order: **tenure guard** (younger than 60 days → `active`, unconditionally, unless `demo-zombie=true`) → **vetoes** (any V1–V8 → `investigate`, score irrelevant) → thresholds (≥80 `zombie-high`, ≥55 `zombie-medium`, else `active`).

Gate 5 — the analyst must fail to kill it. Every non-active verdict goes to the Gemini adversarial pass (§4). A cited contradiction downgrades `zombie-*` → `investigate`. The analyst cannot confirm harder — only soften.

Gate 6 — persistence across runs. One run's verdict is a snapshot, not a sentence. Verdicts are re-derived daily from scratch; Firestore keeps the per-run history and the transition feed shows movement. A borderline project typically *enters* as `investigate` or `zombie-medium` and hardens as evidence accrues (billing months accumulate, the recommender's ~30-day observation matures); any new human action flips it straight back to `active` — a **resurrection**. Upstream consumers are told to require N consecutive zombie-high runs before opening a decommission ticket (§5.4).

Worked example — the real catch from run `run-20260714`: billing 90 (zero spend) × 0.35 + last-accessed 100 (no human action ever) × 0.40 + connectivity 70 (one low-grade signal) × 0.25 = **89** → **zombie-high**, confidence `medium` (billing pillar was only medium-confidence), analyst position `agree` — and the output is still only `recommended_action: "stage decommission: notify owner → 14d scream test → delete"`.

Collector notes learned in production:

- Audit queries must be **project-scoped** (`projects/{id}` log buckets) — the org sink bucket only holds org-level operations.
- Cloud Asset Inventory search is unreliable at org scope with a `project:` query for *fresh* resources (~1–2 h ingestion lag for new KMS/IAM) — collectors scope directly to the project.
- Every collector degrades gracefully: a missing API or empty billing dataset lowers pillar confidence and adds a run note; it never kills the run. Billing was exactly this case until the export table populated.

3.2 Pillar scoring in detail

All three scorers live in `pipeline/scoring.py` — deterministic code, ~90 lines, no LLM involvement. Shared conventions: **0 = alive, 100 = deadest**; a missing signal scores neutral (50) with `unknown` confidence, never deadly.

Billing (weight 0.35) — first matching row wins:

Input state	Score	Confidence
No billing export data for the project	50 (neutral)	unknown
Zero spend in the 90-day window	90	medium
Spend present	`min(100, 0.8 × (holding% + flat bonus))`	high

- **holding%** — the share of spend on pure holding costs, matched two ways per the `holding_cost_*` lists in `policy.yaml`: SKU keywords for holding SKUs inside mixed services (disks, static IPs, snapshots), plus services that are holding-cost in their entirety (Cloud Storage, DNS, KMS key storage). A project spending $400/month entirely on a disk and a static IP scores higher than one spending $4 on compute.
- **flat bonus** — +30 when the month-on-month variance coefficient is < 0.05 across ≥ 2 billed months. Perfectly flat spend is the signature of a corpse paying rent; *low* spend alone is not.
- The **×0.8 damping** means billing without the flat bonus tops out at 80, and only the maximal case (100 % holding *and* dead flat) reaches 100.
- Zero spend deliberately gets 90 at only `medium` confidence — "either dead or free-tier" is genuine ambiguity, and the analyst sees it phrased that way.

Last accessed (weight 0.40) — evaluated in strict order, first match wins:

Condition	Score	Confidence
Audit-log signal unavailable	50 (neutral)	unknown
No human admin action anywhere in the 400-day retention	100	high
Only provisioning-era action (within 48 h of project creation)	90	high
Last human action older than the 90-day window	85	high
Human action inside the window	`age ÷ 90 × 60`	high

- The perfect 100 is reserved for **never touched by a human** — the strongest single zombie signal the tool has. "Long dead" (85) and "only ever provisioned" (90) rank just below it.
- The inside-window ramp is linear and **caps at 60**: an action today scores 0, 45 days ago ≈ 30, 89 days ago ≈ 59. Any human activity in the window pulls hard toward `active`.
- Provisioning grace is computed as *project age − action age ≤ 2 days*, i.e. the action happened in the project's first 48 hours — a forgotten project's only human touch is its own creation.

Connectivity (weight 0.25):

Condition	Score	Confidence
Any hard veto tripped (V1–V8)	0	high

Condition	Score	Confidence
Low-grade surface found (AR repos, KMS keys below veto level)	70	medium
No cross-project surface detected	95	high
— each *incomplete* veto check	−3, capped at 85	one notch lower

- A tripped veto scores the pillar **0 — maximally alive** — which looks inverted until you remember the veto already forces `investigate`. The pillar stays honest ("blast radius exists") instead of double-counting the veto into the deadness score.
- The incomplete-check rule (−3 each, cap 85, confidence downgraded) is the fix from the V7 lien-check incident: a transiently failing check now *shaves* the real signal instead of overwriting it with a constant that flattened distinct projects onto one identical score.

Combination. Weighted sum, rounded (`0.35 / 0.40 / 0.25`); overall confidence is the worst pillar (any `unknown` → `low`; any `medium`/`low` → `medium`; else `high`). Two live checks of the arithmetic:

- Husk seed, billing export still empty: `50×0.35 + 100×0.40 + 95×0.25 = 81.25 → 81` — exactly the run-20260715 verdict.
- The real catch (`travel-imaginator-app`): `90×0.35 + 100×0.40 + 70×0.25 = 89` — zombie-high.

Note the deliberate consequence of the caps and weights: **no single pillar can produce a zombie verdict alone.** The largest one-pillar contribution is last-accessed at 100 × 0.40 = 40, well short of the zombie-medium threshold (55) — a zombie verdict always requires at least two pillars agreeing the project is dead.

3.3 The second axis: delete-risk profile

Deadness and deletion safety are **different questions**, and conflating them is how tools end up either deleting things they shouldn't or refusing to flag things they should. Every deep-assessed project therefore carries two orthogonal results:

- **`verdict` / `score`** — *should we consider deleting this?* (deadness)
- **`delete_risk`** — *can we delete it safely?* (impact profile: `low` = no impact expected, `medium` = possible impact, `high` = certain/likely impact)

The naming reads inverted until it clicks: a **zombie-high with delete_risk=low is the ideal decommission candidate** — thoroughly dead *and* verified safe to remove. The mapping is deterministic (worst signal wins), thresholds in `policy.yaml` under the `delete_risk` block:

Signal	Risk
Any veto (except a dormant V1)	**high** — structural external dependency
V1 grant with no SA authentication in 90 d (Policy Analyzer)	medium — dependency exists but is dormant

Signal	Risk
Any veto check incomplete (`unknown` evidence)	medium — can't rule impact out
Low-grade connectivity surface (AR repos, unbound API keys, KMS keys below veto level)	medium
Runtime activity above thresholds — API requests, instance CPU, NIC traffic	medium — something still talks to it
Data-access events in window (any principal)	medium
Degraded billing signal	medium
All checks complete, clean and quiet	**low**

Machine heartbeat feeds this axis only, never deadness. The `heartbeat` seed proves the split: an SA-driven cron writing to GCS makes a project *risky to delete* (delete_risk medium — traffic exists) while leaving it every bit a zombie (no human, pure holding costs). Runtime signals come from Cloud Monitoring (consumed-API request counts, instance CPU, NIC byte counters — VPC Flow Logs are only used where enabled, which is rare in dev tenancies; the fallback is documented in the evidence, not hidden) plus a Data Access audit-log sample (only services with Data Access logging enabled appear — BigQuery has it by default — so absence is weak evidence and capped at `medium` confidence). The data-access sensor excludes its own reflection — a lesson from the first live run, where the pipeline's daily reads plus Google's platform scanners (SCC, web security scanner) put "100+ recent data-access events" on every project in the org, including stone-dead husks. Two filters resulted: the tool's own SAs and Google-managed service agents are dropped, and only `DATA_READ` / `DATA_WRITE` permission types count — `ADMIN_READ` is control-plane traffic (`GetProject`, `GetIamPolicy`: Terraform refreshes, console browsing, org scanners) that says nothing about anyone *using* the project's data. Humans and customer workload SAs reading or writing actual data count.

DR / seasonal use is the hard residual: a disaster-recovery project is *supposed* to look dead until the one day it isn't. Three mitigations, honestly bounded: (1) the human-activity lookback is the full 400-day Admin Activity retention — an annually-exercised DR project shows admin actions well inside that; (2) billing seasonality analysis activates once the export holds ≥ 6 months of history (13-month recurrence is the goal; a young export emits an explicit "seasonality analysis unavailable" note and per-project evidence rather than silently passing); (3) the portal records **typed, expiring owner attestations** — `no-dr-seasonal-use` (90-day expiry) puts a human on record for exactly the window automation can't yet see. Attestations annotate the assessment; they never lower the automated risk rating.

Each candidate also gets a **full Cloud Asset Inventory dump** — the "what exactly dies if we delete this" artifact: counts by type in `results.json` (`resources_summary`), full listing at `runs/<date>/inventory/<project_id>.json` in the results bucket.

4. Gemini: analyst, not sensor

The LLM (Vertex AI, `gemini-2.5-flash`, temperature **0.0**) never detects anything and never decides anything. The deterministic collectors run first; Gemini receives the finished evidence and the policy engine's pillar conclusions, then must:

1. Make the strongest honest **case for** the zombie verdict,
2. Argue **against itself** — what would make deletion harmful?
3. Land a position: `agree`, `challenge`, or `insufficient_evidence`, with every claim citing evidence IDs. It may only `challenge` by naming a specific contradicting evidence ID.

A challenge can only move a verdict in the **safer** direction (`zombie-*` → `investigate`). The analyst can never upgrade toward deletion. Calls retry on 429/5xx with exponential backoff; on persistent failure the verdict stands on deterministic evidence alone, flagged `insufficient_evidence`.

Determinism matters for trust: temperature 0.0 plus pre-settled pillar conclusions came from a live incident where identical evidence produced flip-flopping verdicts across runs at temperature 0.2.

4.1 Which model, and how to upgrade it

The analyst model is **configuration, not code**: `analyst.model` in `config/policy.yaml` (currently `gemini-2.5-flash`, Vertex AI, `us-central1`, temperature 0.0). Flash was chosen deliberately for the demo: the analyst's job is bounded — argue both sides of *pre-collected, pre-scored* evidence and cite it — and the call volume is small (only non-active candidates reach the analyst), so a fast, cheap model fits.

Swapping to a more capable model (e.g. `gemini-2.5-pro`, or a newer generation as they release) needs **no code or infrastructure change**:

1. Edit `analyst.model` in `config/policy.yaml` and bump `policy_version` — model choice is part of assessment policy, and the bump makes the change visible in every subsequent `results.json`.
2. Rebuild the images (`gcloud builds submit`). Jobs pull `:latest`, so the next scheduled run picks it up — nothing to redeploy.
3. Check the model is served in the configured Vertex region (`analyst.location`) before rolling; that is the only compatibility constraint. The pipeline SA's `roles/aiplatform.user` grant covers any Vertex model.

What an upgrade does and doesn't change: a stronger model raises the *quality of the adversarial challenge* — subtler contradictions caught, better-argued cases — but the safety envelope is identical by construction. Verdict logic stays deterministic in the policy engine, the analyst still cannot harden a verdict (only `zombie-*` → `investigate`), and every per-project record carries the model that produced its analyst opinion (`analyst.model` in `results.json`), so runs remain comparable and attributable across model changes.

5. Output contract — what upstream systems ingest

Two artifacts per run, one source of truth. `results.json` is canonical; `report.md` is *rendered from* the JSON, never generated in parallel. Any system that acts on verdicts — including the upstream deletion workflow — ingests the JSON. The portal is just a demo skin over the same data.

5.1 Where results land

Every run writes four objects to the results bucket:

Path	Content type	Purpose
`gs://sdx-zombie-results/runs/<YYYY-MM-DD>/results.json`	`application/json`	Canonical, immutable per-day record
`gs://sdx-zombie-results/runs/<YYYY-MM-DD>/report.md`	`text/markdown`	Human report rendered from the JSON
`gs://sdx-zombie-results/runs/latest/results.json`	`application/json`	Stable "current state" pointer (overwritten)
`gs://sdx-zombie-results/runs/latest/report.md`	`text/markdown`	Ditto
`gs://sdx-zombie-results/runs/<YYYY-MM-DD>/inventory/<project_id>.json`	`application/json`	Full CAI resource listing per deep-assessed candidate — attach to the decommission ticket

Firestore (in `sdx-demos-zombie-dev`) mirrors the run for querying: `runs/{run_id}` (summary + verdict counts + transitions), `projects/{project_id}` (latest verdict) with subcollection `history/{run_id}` (one doc per run — the audit trail), and `acks/…` (typed, expiring owner attestations — §3.3).

5.2 results.json schema (`schema_version: "1.1"`)

Top level (one run):

Field	Type	Notes
`run_id`	string	`run-YYYYMMDD-HHMMSS-<hex6>` — cite this in change tickets
`timestamp`	string	ISO-8601 UTC
`org_id`	string	Assessed organization
`policy_version`	string	The policy file that produced these verdicts (e.g. `2026-07-17.2`)
`observable_window_days`	int	Signal window (90)

Field	Type	Notes
`population` / `deep_assessed`	int	Projects triaged / deep-assessed
`projects`	array	One assessment per project (below)
`notes`	array of string	**Degraded-signal notices** — e.g. billing export empty. Upstream MUST treat verdicts from a degraded run as lower-confidence
`schema_version`	string	Bump = breaking change; pin your parser

Per project (`projects[]`):

Field	Type	Notes
`project_id` , `project_number`	string	Both stable identifiers — key on `project_number` if your CMDB does
`display_name` , `folder` , `labels` , `create_time`		CRM metadata; `labels["demo-zombie"] == "true"` marks seeded demo projects — **exclude from real workflows**
`owner_of_record`	string	Best-effort human owner from IAM/audit trail — the notify target
`verdict`	enum	`zombie-high` | `zombie-medium` | `investigate` | `active`
`score`	int 0–100	Weighted deadness score, 100 = deadest
`confidence`	enum	`high` | `medium` | `low` | `unknown`
`tenure_guarded`	bool	Too young to be scored a zombie
`delete_risk`	enum	`low` | `medium` | `high` | `""` (not deep-assessed) — the second axis (§3.3): can we delete safely?
`delete_risk_reasons`	array of string	Why the risk landed where it did (worst signal wins)
`runtime_activity`	object	Monitoring sample: `{api_requests, api_top_services, cpu_mean, cpu_max, net_bytes, checks_failed[]}` — delete-risk input only
`resources_summary`	object	CAI inventory: `{total, by_type, truncated, confidence}` ; full listing in the per-project inventory object (§5.1)
`attestation`	object	Unexpired owner attestation from the portal: `{type: known-keep|no-dr-seasonal-use, by, note, created, expires}`

Field	Type	Notes
`vetoes`	array	`{code: "V1".."V8", description, evidence_id}` — non-empty ⇒ verdict is `investigate`
`pillars`	object	`billing / last_accessed / connectivity → {score, confidence, rationale}`
`evidence`	array	`{id, pillar, summary, detail, confidence}` — `id` (e.g. `E-billing-<project>`) is what analyst citations point at
`analyst`	object	`{case_for_zombie, case_against_zombie, final_position: agree|challenge|insufficient_evidence, challenge_reason?, citations[], model, error?}`
`recommended_action`	string	**Advice only** — nothing in the tool executes it
`best_practice_findings`	array	Side-channel hygiene findings, not verdict inputs

Abbreviated real example (the tool's first genuine catch; identifiers sanitized):

```
{
  "project_id": "travel-imaginator-app",
  "project_number": "123456789012",
  "owner_of_record": "owner@example.com",
  "verdict": "zombie-high",
  "score": 89,
  "confidence": "medium",
  "tenure_guarded": false,
  "vetoes": [],
  "pillars": {
    "billing":       {"score": 90,  "confidence": "medium",
                      "rationale": "zero spend in window (either dead or free-tier)"},
    "last_accessed": {"score": 100, "confidence": "high",
                      "rationale": "no human admin action in retained logs"},
    "connectivity":  {"score": 70,  "confidence": "medium",
                      "rationale": "1 low-grade connectivity signal(s)"}
  },
  "evidence": [
    {"id": "E-lastaccess-travel-imaginator-app", "pillar": "last_accessed",
     "summary": "no human principal in Admin Activity logs", "confidence": "high"}
  ],
  "analyst": {"final_position": "agree",
              "citations": ["E-lastaccess-travel-imaginator-app"]},
  "recommended_action": "stage decommission: notify owner → 14d scream test (disable APIs) → delete (30d
        soft-delete)"
}
```

5.3 Ingestion paths (pick one)

1. **GCS pull (recommended for batch):** grant the upstream system's SA `roles/storage.objectViewer` on `sdx-zombie-results` and read `runs/latest/results.json` after 06:30 Sydney daily (run takes ~5 min from 06:00). Dated paths give you replayable history for free.

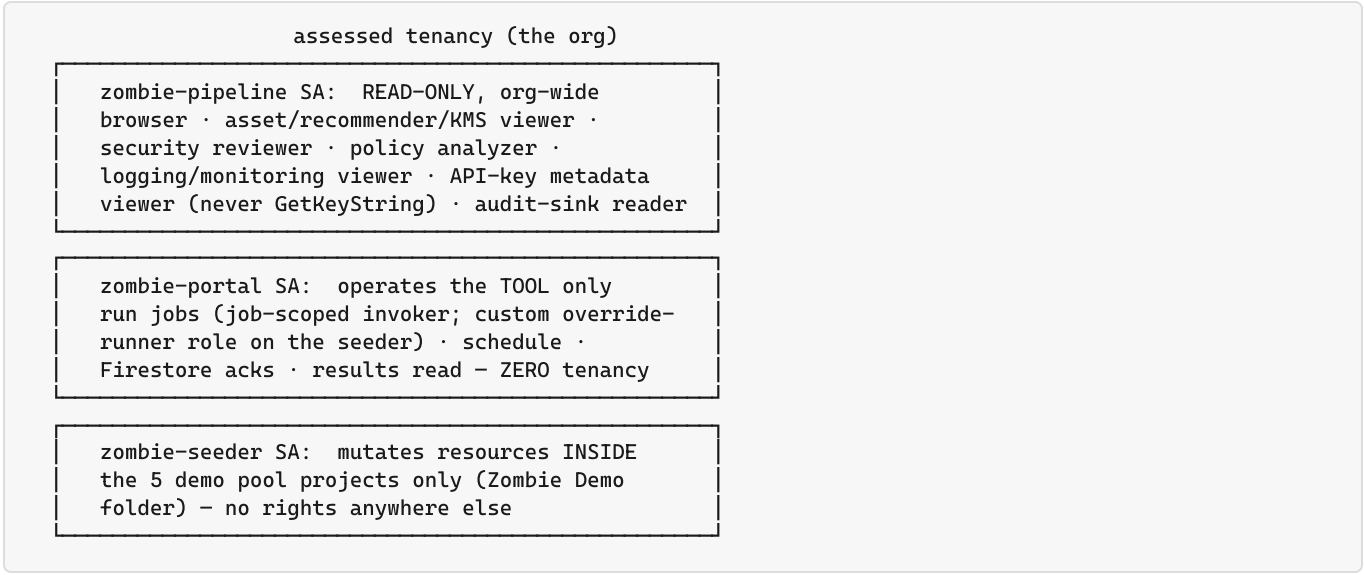
2. **Results API (pull, per-project queries):** the portal's API behind IAP — `GET /api/runs/latest`, `/api/runs/{date}`, `/api/projects/{id}/history`, `/api/report/latest`. Programmatic access = standard IAP OIDC: grant the caller SA `roles/iap.httpsResourceAccessor` on the service and send an identity token. `/history` is the endpoint a deletion system should care about most (see 5.4).
3. **Webhook push (event-driven):** set `WEBHOOK_URL` on the pipeline job and it POSTs on every run **that produced verdict transitions**: `{"run_id": "...", "transitions": [{"project_id", "from", "to", "date"}]}`. Fire-and-forget (10 s timeout, failures logged, never block the run) — use it as a trigger to pull the canonical JSON, not as the payload of record.

5.4 Contract for an upstream deletion system

The tool is deliberately advice-only; if you build the deleter upstream, these are the rules the verdicts are designed around:

- **Act only on `zombie-high` with `delete_risk: low`** (and `zombie-medium` only if your risk appetite says so). **Never** act on `investigate` — a veto means deletion has known blast radius — and never on anything with `tenure_guarded: true` or the `demo-zombie` label. `delete_risk: medium` means resolve the listed reasons (or collect an owner attestation) first; `high` means the dependency is structural — treat as investigate.
- **Require persistence, not one snapshot:** verdicts migrate as evidence accrues (and projects *resurrect* — a new human action flips a zombie back to active). Gate on N consecutive runs at zombie-high via `/api/projects/{id}/history` or the Firestore `history` subcollection before opening a decommission ticket. Suggested N = 7 daily runs.
- **Check run `notes`:** a degraded-signal run (e.g. billing export unavailable) weakens the billing pillar org-wide — don't let those runs count toward the persistence gate.
- **Honor attestations:** `acks/{project_id}` in Firestore (also `GET /api/acks`) records a typed, expiring owner attestation — `known-keep` ("seen it, leave it": skip these while unexpired) or `no-dr-seasonal-use` (covers the seasonality window automation can't yet verify). Unexpired attestations also surface per project in `results.json` as `attestation`.
- **Keep the human step:** the intended flow is notify `owner_of_record` → 14-day scream test (disable APIs — reversible) → delete, with GCP's 30-day soft-delete as the final undo. Cite `run_id` + evidence IDs in the ticket so every deletion traces back to the exact evidence that justified it. Never blind billing-disable.

6. Governance: three service accounts, one hard line



Where the SAs live vs where their grants attach

A common misreading: the pipeline SA is *not* in a different project. All three service-account **identities** live in the tool project — scope comes from where each **IAM grant attaches** in the resource hierarchy, not from where the identity is homed. The pipeline SA reaches the whole org because its viewer roles are bound at the org node and inherit downward; the seeder is the same pattern one level down, folder-scoped:

```
org: sequin.au ◂— zombie-pipeline@: 10 viewer-only roles
 │                (browser · cloudasset.viewer · recommender.viewer ·
 │                 iam.securityReviewer · policyanalyzer.activity-
 │                 AnalysisViewer · cloudkms.viewer · monitoring.viewer ·
 │                 logging.viewer · serviceUsageViewer · apiKeysViewer)
 │                – inherited by every folder and project below:
 │                  this inheritance IS the "assessed tenancy" reach
 │
 ├─ Enterprise / Dev
 │   └─ sdx-demos-zombie-dev  (tool project)
 │       ● all three SA IDENTITIES are homed here:
 │         zombie-pipeline@ · zombie-portal@ · zombie-seeder@
 │       ◂— pipeline@: datastore.user · aiplatform.user ·
 │            bigquery.jobUser · logging.logWriter
 │            + objectAdmin on gs://sdx-zombie-results (bucket-scoped)
 │            + dataViewer on the billing_export dataset
 │       ◂— portal@: cloudscheduler.admin · run.viewer ·
 │            datastore.user · logging.logWriter
 │            + JOB-scoped (runtime.tf): run.invoker on the pipeline
 │              job · zombieJobOverrideRunner on the seeder job
 │       ◂— seeder@: datastore.user · logging.logWriter
 │
 ├─ Sandbox
 │   └─ Zombie Demo folder ◂— seeder@ (folder-scoped, stops here):
 │       │                    editor · projectIamAdmin · cloudkms.admin
 │       │                    · serviceUsageAdmin · folderViewer
 │       ├─ sdx-zh-husk-alpha ──┐
 │       ├─ sdx-zh-husk-beta    │  TF-managed seed pool (billing linked
 │       ├─ sdx-zh-heartbeat    │  by the human `terraform apply` –
 │       ├─ sdx-zh-veto-sa      │  Argolis: no SA can hold billing)
 │       └─ sdx-zh-veto-cmek ──┘
 │
 ├─ everything else – the assessed tenancy: reached ONLY via the
 │  inherited org-level viewer grants, nothing writable
 │
 └─ gs://sdx-audit-logs (org audit sink) ◂— pipeline@: objectViewer
```

The asymmetry is deliberate: the SA with the widest *reach* (pipeline, org-wide) has the weakest *verbs* (viewer-only), and the SA with the strongest verbs (seeder, `editor` / `kms.admin`) has the narrowest reach (one demo folder). The portal SA has no tenancy grants at all.

Every delete requires a human — by construction

Path	Guarantee
Pipeline vs tenancy	IAM: viewer-only roles org-wide. Code: zero mutating API calls — verified by audit. Verdicts and `recommended_action` are **advice strings**, nothing executes them.
Real-project decommission	Entirely outside the tool: a human admin runs **notify owner → 14-day scream test (disable APIs) → delete**. GCP's 30-day project soft-delete is the final undo. Never blind billing-disable.
Demo teardown	Fires only from a human clicking the IAP-gated portal button (plus a browser confirm). Strips resources *inside* the pool projects; cannot touch the projects themselves.

Path	Guarantee
Seeder SA	Holds **no** `projectCreator` / `projectDeleter` anywhere (revoked 2026-07-15 — the pool made them unnecessary). Pool project create/delete is Terraform, applied by a human.
Unattended execution	The only scheduled job is the read-only pipeline. Nothing on a schedule can delete anything.

Portal access itself is IAP-gated (`iap.httpsResourceAccessor`, granted only to the human operator principals); Cloud Run's ingress accepts only the IAP service agent.

7. Demo seeding

Structural zombie signals demo immediately, but billing trends need days and the Unattended Project Recommender needs ~30 days — so demos are seeded ahead of time. A **Terraform-managed pool** of five `sdx-zh-*` projects in a dedicated Zombie Demo folder spans the verdict spectrum:

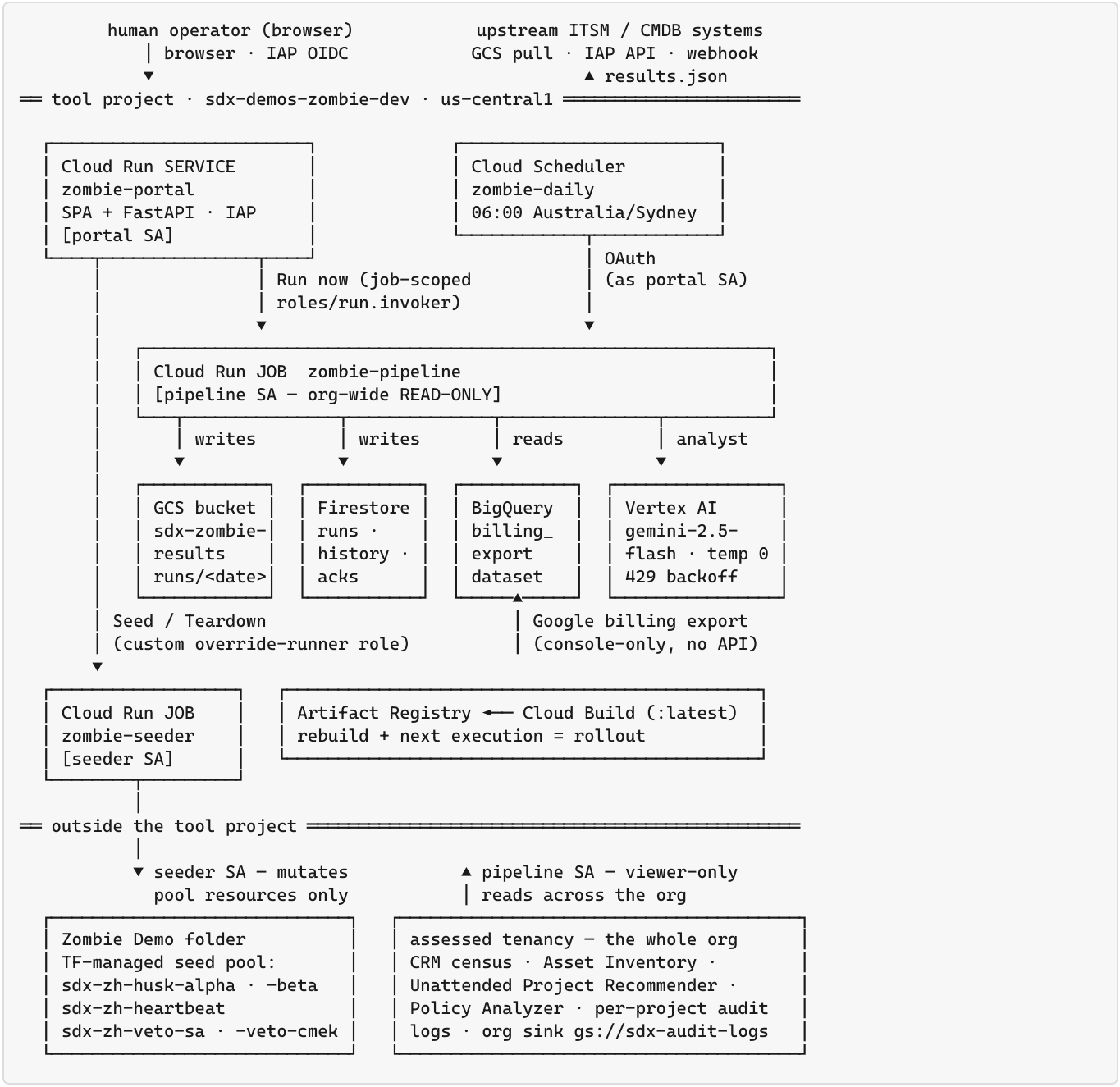
Seed	Persona	Expected verdict
`sdx-zh-husk-alpha`	Corpse paying rent: orphan disk, static IP	zombie-high
`sdx-zh-husk-beta`	Bucket of forgotten artifacts	zombie-high
`sdx-zh-heartbeat`	Dead but twitching: cron → Pub/Sub pulse	zombie-medium
`sdx-zh-veto-sa`	Its SA is used by another project	investigate (V1)
`sdx-zh-veto-cmek`	Its CMEK key encrypts external data	investigate (V2)

The seeder job populates/strips resources inside the pool (`MODE=seed|teardown`, driven by portal buttons — passed as a per-execution env override, which is why the portal SA carries a custom `zombieJobOverrideRunner` role on the seeder job: `run.jobs.runWithOverrides` is not part of `roles/run.invoker`). Seeding while the pool is populated returns `409 — teardown first`; the seeder enforces the same guard itself. The pool is TF-managed because the Argolis billing account is Google-held — no service account can ever link billing at runtime, which is itself a nice governance property. The `demo-zombie=true` label bypasses the tenure guard and badges rows as seeds in the portal; the analyst prompt discloses the label so it assesses the constructed evidence at face value.

8. Infrastructure

8.1 Deployment topology

Everything below the double rule lives in the tool project; the pipeline's only relationship to the assessed org is viewer-only reads, and the seeder's only write surface is inside the demo pool.



8.2 Terraform

Everything is Terraform, **in this repo** at `terraform/` (the `project-factory` module is vendored under `terraform/modules/`). The GCS state backend and deployment-specific values are supplied via untracked files — `backend.hcl` and `terraform.tfvars`, examples committed alongside:

- **`main.tf`** — tool project + APIs, Zombie Demo folder, three SAs and their IAM (org-wide read-only grants, folder-scoped seeder grants), seed-project pool, Firestore, results bucket, Artifact Registry, billing export dataset.

- `runtime.tf` — both Cloud Run jobs, the portal service (`iap_enabled`, google-beta provider), the daily scheduler, job-scoped invoker + IAP bindings, and the `zombieJobOverrideRunner` custom role (seed/teardown env overrides need `run.jobs.runWithOverrides`). Imported from the live gcloud-deployed shapes 2026-07-15; plan is zero-drift.

Images build via Cloud Build (`cloudbuild.yaml`) into Artifact Registry and deploy as `:latest` — TF owns the *shape*, a rebuild + next execution is the rollout. One manual step remains manual by necessity: enabling the BigQuery billing export has no API/gcloud/TF path (console-only, verified).

9. Known limitations & roadmap

- **Billing export lag** — the BQ export lands hours after enablement and backfills ~5 days to the start of the previous month; the billing pillar runs at `unknown` confidence for projects without rows until data accrues.
- **CAI ingestion lag** — fresh KMS/IAM resources take ~1–2 h to appear; V2 sweeps on brand-new seeds can miss until then.
- **Recommender warm-up** — Unattended Project Recommender needs ~30 days of observation; triage leans on billing/structural signals meanwhile.
- **Seasonality maturity** — billing-recurrence analysis needs ≥ 6 months of export history (13 ideal) and says so until then; VPC Flow Logs are only used where enabled. Both bounded honestly in §3.3 and `PHASE1-GAP-ANALYSIS.md`.
- **Future: Gemini CLI + Cloud Assist MCP** — the analyst currently uses the Vertex `google-genai` SDK directly; the original spec's Gemini CLI + remote MCP path (Cloud Assist, gcloud, BigQuery MCPs, Agent Actions disabled) was deferred as the riskiest dependency for a headless job, and remains the planned enhancement.
- **Production hardening** — per-customer deploys would drop the portal for programmatic consumption, add org-policy guardrails around the demo folder, and scale stage-2 fan-out (the two-stage split already bounds cost: triage is O(all projects) but cheap; deep assessment is O(candidates)).